RESEARCH PROGRESS / 2026.10.22

AIハーネスの引き継ぎで何を残すべきか

会話履歴と実行中の作業状態を分け、
どの情報を残すと仕事を継続できるか調べる。

対象

状態を変える複数工程の仕事

現在地

通常方式の比較から、引き継ぎの条件比較へ

投稿目標

10月31日 IEICE・4ページ

必要なのは、複数の工程を最後まで終えること

調べる・計算する・変更する・確かめる。
一つずつ正しくても、全体の要件が残ることがある。



情報を集める

文書・要求・観測

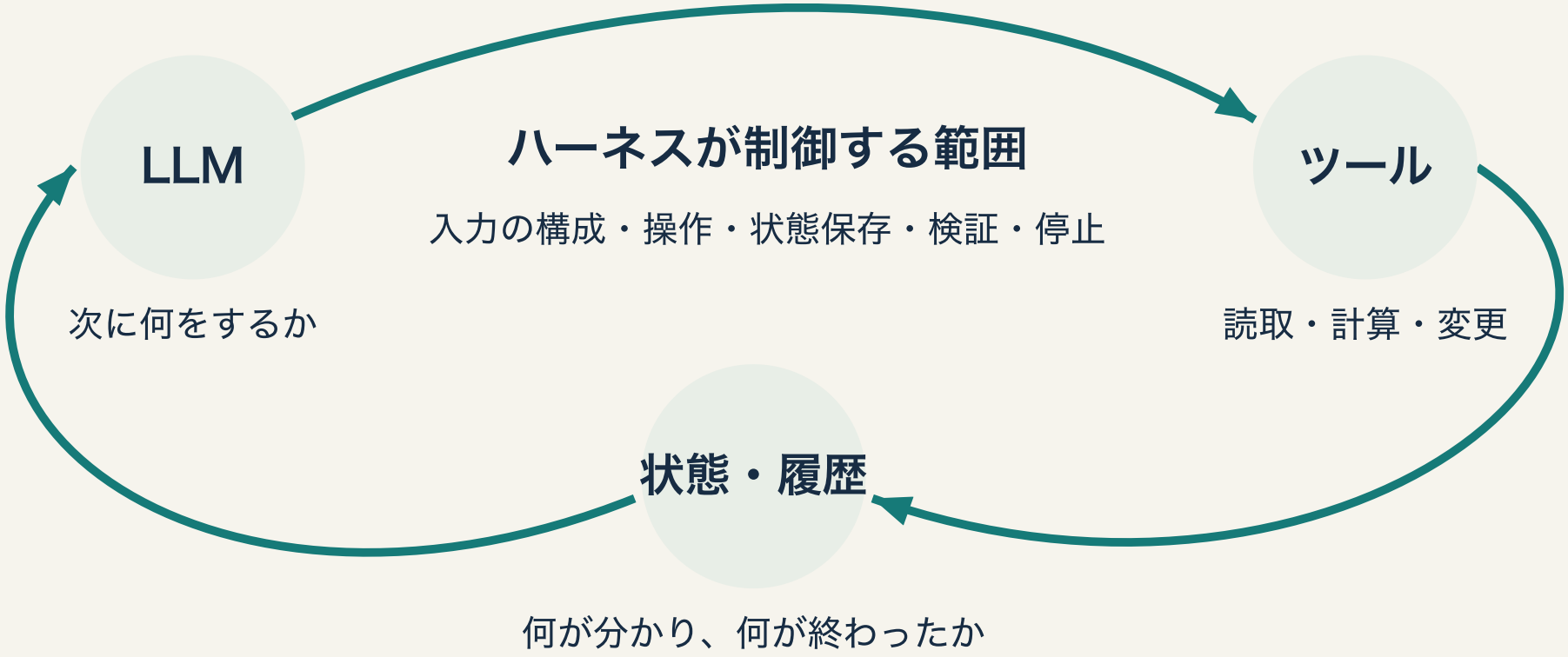
処理して変更する

コード・API・ファイル

完了を確かめる

要件と最終状態

モデルの外側にも、結果を変える設計がある



作業状態が残れば、渡す会話は減らせるか

01
完了できるか

要件を満たす最終状態を、実行とは別に評価する。

02
費用に見合うか

記憶生成・調整・失敗も含め、トークンと時間を測る。

03
なぜ変わったか

指示、情報保持、操作形式、予算を切り分ける。

> 目標：条件をそろえた比較で、必要な機構と不要な追加処理を示す。

再編の効果と、引き継ぎの効果を分けたい

当初の新提案候補・未評価

作業間依存に応じた実行単位の再編

分担の継続・情報共有・中央調整・作業集約を、同じ途中状態から比較する。

現在の実験

担当を変えず、残す状態を比較

再編時には会話や変数の持ち方も変わる。まずその影響を、同じ地点・担当・継続予算で調べる。

> 現在の結果だけで、マルチエージェント再編の有効性は主張できない。

既にある機能を、独自の技術と数えない

先行研究・仕組み	既に扱われていること	今回の扱い
CodeAct / ICML 2024	Pythonで行動を統合し、実行結果から修正	通常コード型の比較相手
FOCUS / 著者公開原稿	今後の作業に必要な情報の選択・保持	共通環境向けに接続した比較
SelfCompact / 著者公開原稿	コンテキスト整理とキャッシュ費用	費用の差を新原理としない
CaveAgent / Runtime Contracts	状態の保持・共有、一時移管・通知の比較	保持と可視性だけでも既出

独自方式の有効性は、まだ確立していない

実装・確認済み

比較と原因分析の基盤

公開APIの接続／状態保存と再生／全費用の記録／独立評価／コード実行型。

いずれも、それだけを新規性とはしない。

研究仮説・未確立

持っている作業状態に応じて、引き継ぐ履歴を変える

保持状態と履歴削減の関係が残れば、選択規則を固定し、強い基準・同じ予算・未調整課題で確認する。

> 今の成果は検証基盤と切り分け。新規性・採録可能性の証明はこれから。

同じモデル・同じ環境で、構成差を比較する

計算機	Mac Studio / M3 Ultra 256 GiB RAM
モデル	MiniMax-M3 / IQ4_XS 重み約229.7 GB・版固定
推論	llama.cpp / 64K context KV q8・1スロット

環境	AppWorld 0.1.3.post1 復元可能な模擬アプリ
全体実行の上限	45万token・80生成・30分
引き継ぎ後の上限	12万token・16生成・8分
評価	終了後の別プロセス 正解をモデルへ戻さない

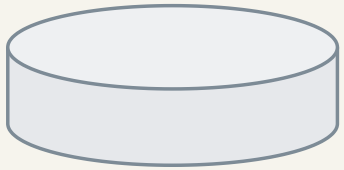
履歴を減らす影響は、変数が残るかで変わるか

共通の途中地点：各課題の４応答後

会話履歴
全部 ／ 直近１ターン

{ 変数・関数 }

Pythonの作業状態
保持 ／ 一回初期化



環境の状態
全条件で同じ

- **全履歴 ＋ 保持**
- **直近 ＋ 保持**
- **全履歴 ＋ 初期化**
- **直近 ＋ 初期化**

同じ後続予算：16生成・12万token・８分。原記録への再アクセスは全条件で可能。

> 二要因の比較は効果を切り分ける手段。この設計だけを新規性とはしない。

同じ途中状態から、全16分岐を評価する

	会話：全部	会話：直近だけ
変数を保持	● ○ ● ○ 2/4 332,236 token ／ 187 API	○ ○ ○ ○ 0/4 416,022 token ／ 196 API
変数を初期化	○ ○ ● ○ 1/4 316,491 token ／ 176 API	○ ○ ● ○ 1/4 392,787 token ／ 144 API

● 成功 ／ 淡色 未達。各 4 既知課題・1 seed。数値は後続のみ。トークン不明 2 呼出を含むため既知分の下限。

> 履歴と作業状態の組合せを観測。新しい選択方式の有効性は別の検証が必要。

８応答後の前提確認も済ませ、全件結果を待つ

なぜ追加するか

４応答後では、３課題にユーザー変数が２つだけ。状態が蓄積した地点も同じ規則で調べる。

何を固定したか

元の全４軌跡の８応答後。
同じ４条件・16生成・12万token・８分。
失敗した地点だけを選ばない。

> 会話の短縮と変数の保持の効果を測る。担当再編の有効性はまだ直接測っていない。

まだ言えないことを、実験結果の隣に置く

一般性の限界

- 主に 1 モデル・各条件 1 seed。
- 既知の少数開発課題を繰り返し利用。
- 別モデル・別分野への転用は未確認。

技術的主張の限界

- 再編の固有効果は未測定。
- 通常コード型にも提示方法などの実装差。
- 独自方式の新規性・有効性は未確定。

> 未閲覧12シナリオは、方式・閾値・比較条件を固定してから使う。

次は、通常方式でも残る一つの問題に絞る

1

引き継ぎ比較を閉じる

履歴・変数・再取得の関係と、費用・失敗を確認する。

2

選択規則が必要か判断

一律に保持すれば十分か。状態に応じる利益が残るか。

3

未調整課題で確認

差分が成立する規則だけを、同じ予算の基準と比較する。

> 成果の形：新しい制御規則、または条件を切り分けた新しい実証知見。

月末投稿へ、10月23日に中心となる主張を固定する

10/23

主比較・あらまし

主張、最も近い先行研究、主要図表を決める。

10/25

追加確認を終了

必要な対照と限界確認に絞り、機能追加を止める。

10/29–30

４ページ原稿

本文・図・引用・公式様式・著者情報を確認する。

10/31

投稿完了

採録時期は査読に依存。投稿と採録を区別する。

> 現状：投稿準備は高リスク。新規性と確認実験が、日程の最も大きな制約。

作った基盤と、論文に必要な成果を区別する

ここまでに得られたもの

比較可能な実行基盤。
一般指示・完了形式・API上限の切り分け。
普通のコード実行を含む比較相手。

これから得るべきもの

既存方式を超える必要性。
新しい規則または実証知見。
同じ予算・未調整課題での裏づけ。

> 目指すのは、構成が効く条件と壊れる条件を説明できるハーネス研究。

実験を増やす前に、次へ進む根拠を確認する

切り分けたいこと	比較	ここまでの判断
履歴を減らす利益	全履歴／直近／保持／台帳	今回の設定では品質と総費用を両立せず
通常の環境案内の不足	一般指示の前後	同じ６件：2→6成功、別４件：2→3成功
完了形式の影響	同じ状態＋回答あり／省略	既知１課題で合否が変化
API上限の影響	写真整理１件：80／160	最初の80操作は同じ、追加操作で成功
値の受け渡しの負担	個別API／取得から操作までのコード	全４件の結果・監査を確認済み

入力が短くても、システム全体が安くなるとは限らない

保持方法	成功	全役割の既知トークン	実行時間
全履歴	4/4	440,355	21.7分
直近履歴	3/4	627,363	29.1分
FOCUSに基づく保持	2/4	467,047 ＋不明	88.7分
旧・作業台帳	2/4	483,251 ＋不明	84.6分

旧台帳は出典の形式契約に不整合があり、後に修正。
FOCUSは共通環境向けの接続で、原論文の完全再現ではない。

新機構を入れる前に、基準の成績が変わった

同じ６開発課題

従来の指示



2/6

一般指示を補足



6/6

別の４開発課題

従来の指示



2/4

一般指示を補足



3/4

同じモデル・課題・予算。時刻・全ページ・連絡先・完了形式などを一括で案内。

同じ成果物でも、完了APIの形式で合否が変わった

記録済み前半の再生	回答文あり	回答文を省略
一般指示を補う前の操作系列	未達	成功
一般指示を補った後の操作系列	未達	成功

４条件とも、完了直前のDBは同一

公開返却と保存状態のハッシュを照合。
LLM追加０、模擬API要求118。

公開仕様に沿った影響

行動依頼では回答文を省く規則がある。
評価器の不具合や独自手法の効果とはしない。

写真整理の残り15件は、追加操作で完了した

上限80：未達

80 API / 185,739 token



27/42

上限160：成功

97 API / 361,169 token



42/42

■ 移動の返却あり　　淡色：移動の返却なし

コード内で、取得した値を次の操作へ渡す

上：モデルが個々のAPI引数を出力



取得した値を、履歴から次の引数へ

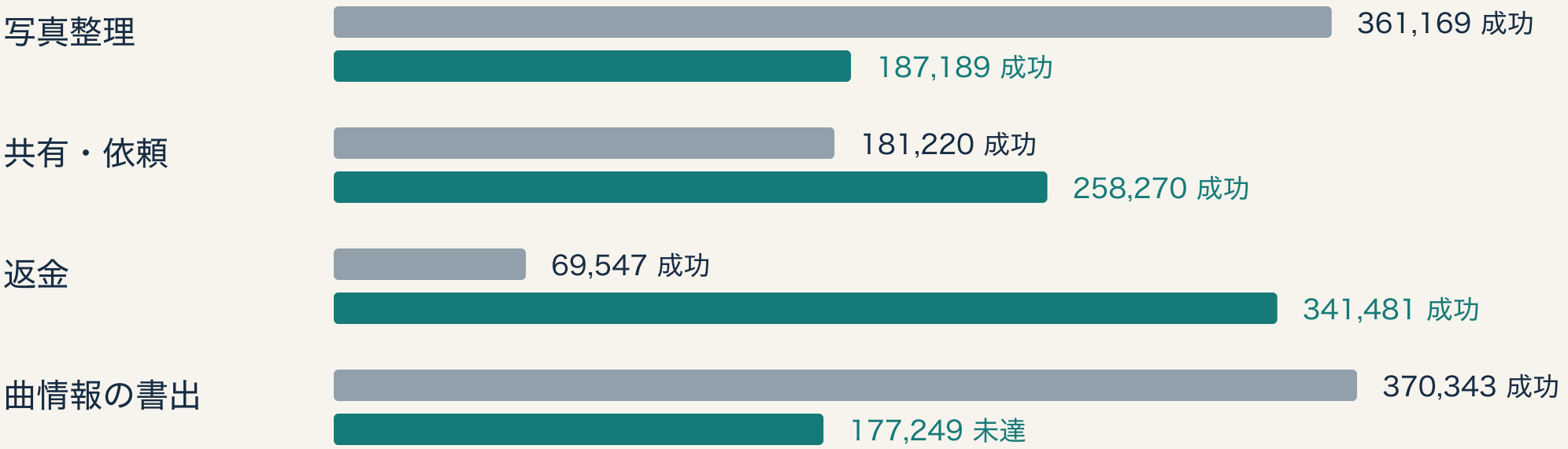
下：コード内で取得→計算→操作をつなぐ



Python内で値を保持。各APIの費用と結果は別々に記録する。

初期接続は3/4。通常API基準の4/4に届かない

全入力＋全出力トークン（小さいほど少ない）



灰：既存API型　青緑：コード実行型。全４件・各１seed。

関数の説明と、変数を残す効果を分ける

先に説明だけを補い、その後に課題内の状態保持を比較。モデル・課題・API上限は維持。

比較で変える場所：プログラムの間の状態

毎回初期化



変数が消える：元観測の再読・再取得・再計算が必要

課題内で保持



変数・関数が残る：取得した値を後の操作へ使える

同じ４課題の全結果を、成功と費用で見る

課題	初期接続	関数説明を補足	説明＋変数保持
写真整理	成功 ／ 187,189	成功 ／ 134,387	成功 ／ 209,299
共有・依頼	成功 ／ 258,270	成功 ／ 218,993	成功 ／ 224,588
返金	成功 ／ 341,481	成功 ／ 81,974	成功 ／ 84,818
曲情報の書出	未達 ／ 177,249	未達 ／ 183,028	未達 ／ 121,330
成功数 / 全費用	**3/4 ／ 964,189**	**3/4 ／ 618,382**	**3/4 ／ 640,035**

数値＝全入力＋全出力トークン。各 1 seedの開発比較。未集計は未実行・実行中・監査待ちを含む。

> 追加の仕組みが必要かを決めるため、通常方式を十分にした状態で比べる。

P27の全比較 ／ P28の全比較 ／ P29の全比較

失敗の観測と、追加機構の必要性は分ける

指示で解消する失敗

元の低成績だけでは、記憶・ゲート・分担変更が必要だとはいえない。

予算で止まる処理

未完了をすべて情報欠落やハルシネーションへまとめない。

正解でも残る未確認

57曲中56曲の詳細取得で正解した例。正解と網羅的な確認は別。

> 改善を確認できた要因から順に閉じ、残る問題にだけ新しい処理を加える。